

Applied NLP

DATA 641

Lecture 5 — Word2Vec; SkipGram

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, ISBN 9781617294631

Word2vec Conceptually

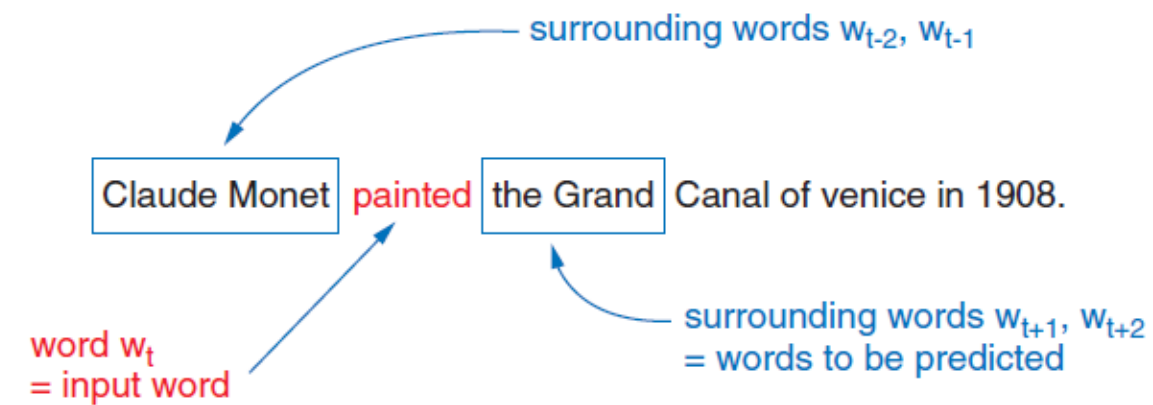
Given a large corpus of text as input, learn to represent the words in a common vector space based on the contexts, they appear in the corpus

- For a given word and the words appearing in its context estimate the vector that best represents the meaning of the word
- Word2vec model starts with a vector of random values and refines the values in that vector by predicting the vector, given the vectors for words in the context
- It does this using a two-layer neural network

SkipGram approach

- **SkipGram:** Predict the context words from the "center" word

Skip-grams are n-grams that contain gaps because you skip over intervening tokens. In this example, you are predicting "Claude" from the input token "painted," and you skip over the token "Monet."

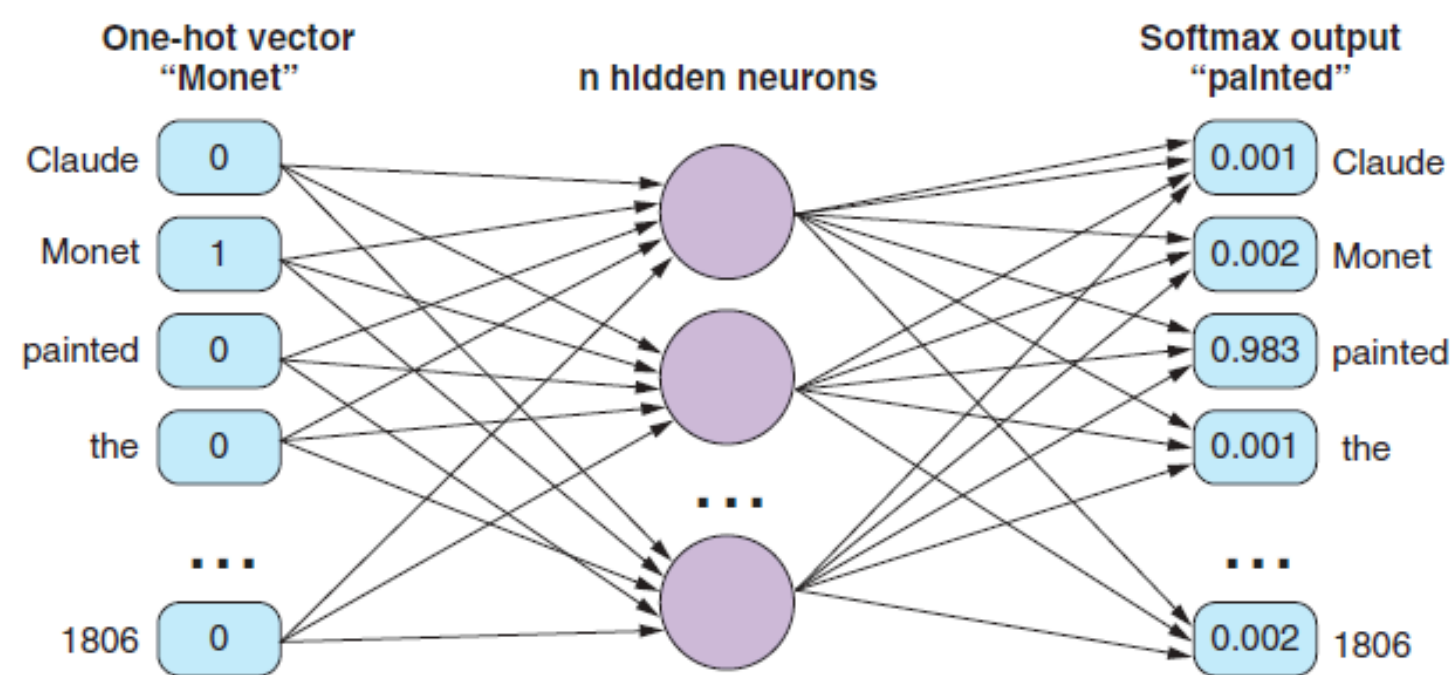
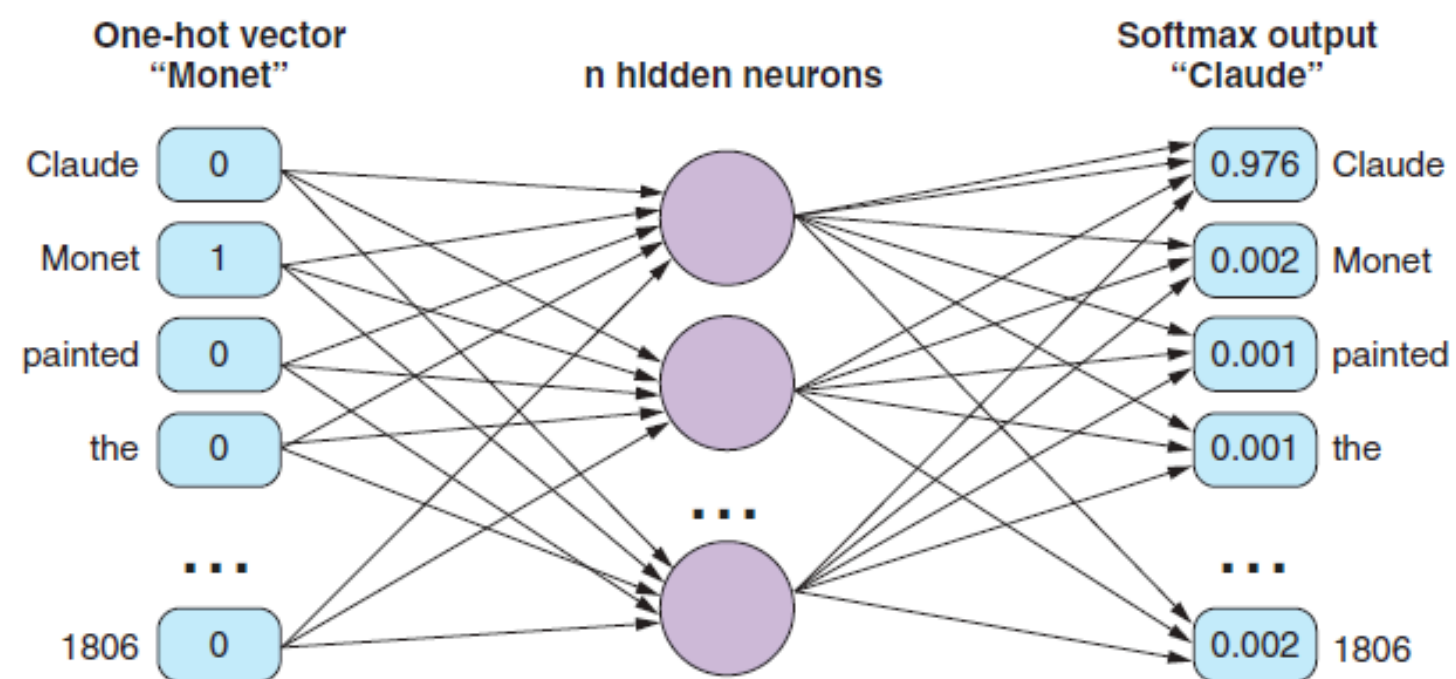


What is Softmax

- The softmax function is often used as the activation function in the output layer of neural networks when the network's goal is to learn classification problems.
- The softmax will squash the output results between 0 and 1, and the sum of all outputs will always add up to 1. That way, the results of an output layer with a softmax function can be considered as probabilities.
- For each of the output nodes, the softmax output value can be calculated using the normalized exponential function:

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (1)$$

How does the network in SkipGram look like?



How does the network learn the vector representations?

In skip-gram, we need to consider the two words (in the window size 2 case) before and after each target word. We would then use a 5-gram tokenizer to turn a sentence

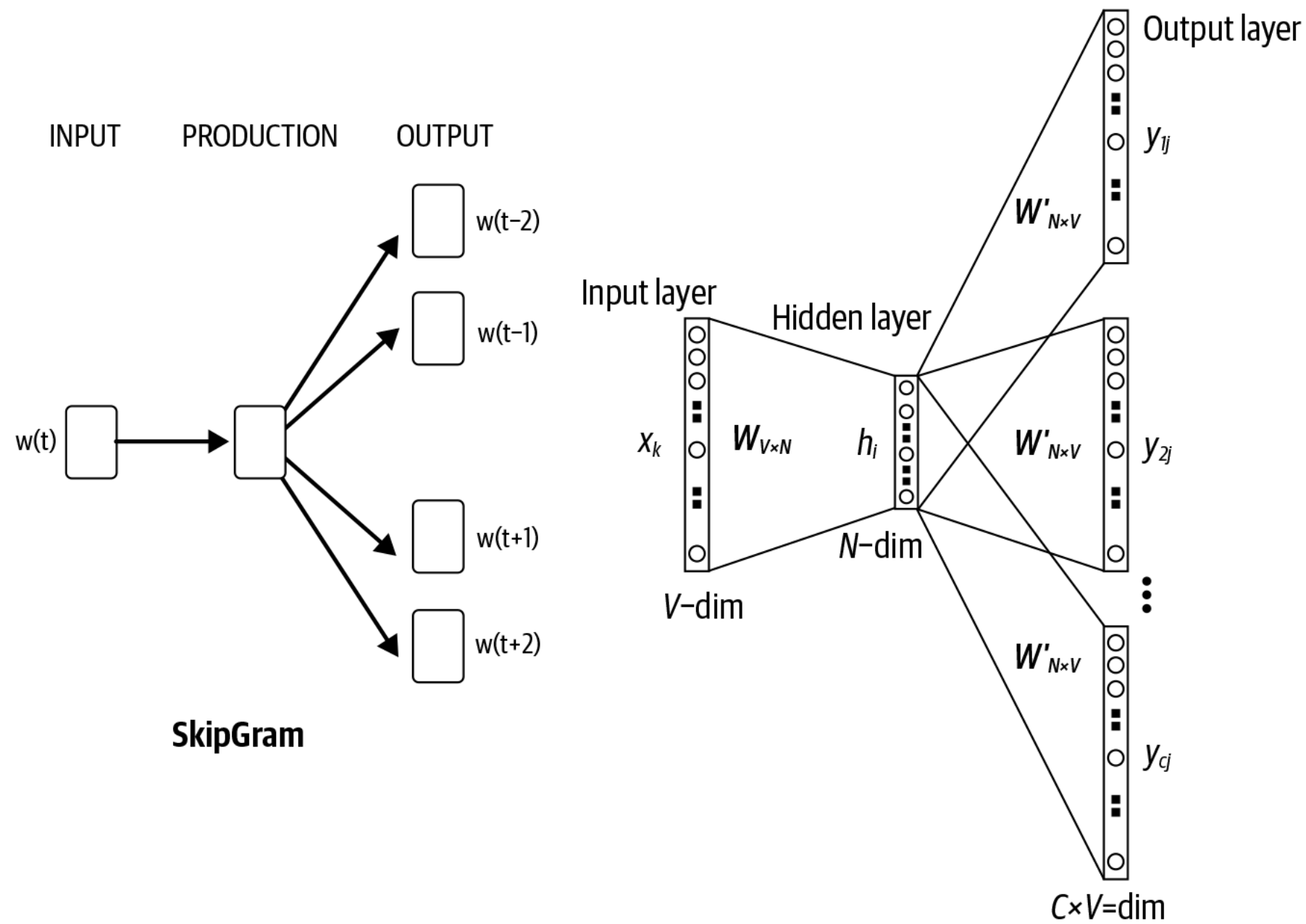
sentence = "Claude Monet painted the Grand Canal of Venice in 1806."

into 10 5-grams with the input word at the center, one for each of the 10 words in the original sentence.

Input word w_t	Expected output w_{t-2}	Expected output w_{t-1}	Expected output w_{t+1}	Expected output w_{t+2}
Claude			Monet	painted
Monet		Claude	painted	the
painted	Claude	Monet	the	Grand
the	Monet	painted	Grand	Canal
Grand	painted	the	Canal	of
Canal	the	Grand	of	Venice
of	Grand	Canal	Venice	in
Venice	Canal	of	in	1806
in	of	Venice	1806	
1806	Venice	in		

The training set consisting of the input word and the surrounding (output) words. In the case of four surrounding words, you would use four training iterations, where each output word is being predicted based on the input word.

Continuous SkipGram Model



- The softmax activation of the output layer nodes (one for each token in the vocabulary) calculates the probability of an output word being found as a surrounding word of the input word.
- The output vector of word probabilities can then be converted into a one-hot vector where the word with the highest probability will be converted to 1, and all remaining terms will be set to 0.
- After training of the neural network is completed, we will notice that the weights have been trained to represent the semantic meaning.
- Each row in the weight matrix represents each word from the vocabulary for your corpus.
- After the training is complete only the weights of the inputs to the hidden layer are used as the embeddings.

Retrieving word vectors

